BILKENT UNIVERSITY

ENGINEERING FACULTY

DEPARTMENT OF COMPUTER ENGINEERING

CS491 - Senior Design Project I

Mergent

Project Specification Document

T2508

Hasan Kutluhan Şıpka, Furkan Emre Kolukırık, Erdem Aydın, Kaan Aydeniz, Oğuzhan Genç

Supervisor: Eray Tüzün

Innovation Expert: Mert Emin Kalender

25.11.2025

Table of Contents

1. Introduction **3**
1.1. Description 3
1.2. High-Level System Architecture & Components 4
1.3. Constraints 5
1.3.1. Implementation Constraints 5
1.3.2. Economic Constraints 5
1.3.3. Sustainability Constraints 5
1.3.4. Time Constraints 5
1.3.5. Privacy Constraints 6
1.4. Professional and Ethical Issues 6
1.5. Standards 6
2. Design Requirements **7**
2.1. Functional Requirements 7
2.1.1. PR-Issue Linkage and Suggestion 7
2.1.2. PR-Issue Alignment Validation 7
2.1.3. AI-Generated Review Comments 7
2.1.4. PR Enrichment with Risk & Impact Insights 8
2.1.5. Vibe Reviewing 8
2.1.6. Post-Review Feedback 8
2.2. Non-Functional Requirements 9
2.2.1. Usability 9
2.2.2. Reliability 9
2.2.3. Performance 9
2.2.4. Security 9
2.2.5. Availability 10
2.2.6. Scalability 10
3. Feasibility Discussions **10**
3.1. Market & Competitive Analysis 10
3.2. Academic Analysis 12
4. Glossary **15**
5. References **16**

1. Introduction

1.1. Description

Code reviews are a vital part of modern software development, ensuring code quality, reliability, and maintainability. However, current pull request (PR) reviews are often slow, inconsistent, and unclear. Issue–PR mismatches waste reviewer time, and the median PR approval in large projects can take 15–20 hours [1]. Human review quality varies, with approximately 32–36% of review comments classified as noisy or irrelevant [2]. Furthermore, due to the lack of structured feedback for reviewers, 21.3% of comments are ultimately dismissed as “Won’t Fix” [1]. As a result, review cycles become longer, rework increases, and developers spend 8–16% of their working week on reviews [1].

To overcome these challenges, Mergent is proposed as an AI-powered intelligent code review assistant designed to make reviews faster, clearer, and more consistent. Mergent enhances the review process through six key phases that together form a comprehensive review pipeline. It begins with Issue Linkage and Alignment, where the system detects or generates issue links and ensures each PR truly addresses its intended issue. During the AI Review Comments phase, Mergent instantly provides precise, context-aware, and actionable feedback. In the PR Enrichment stage, it highlights risky modules, flags bug-prone areas with history of recurring issues, and assigns a simple risk score to guide reviewers’ attention.

The Vibe Reviewing phase introduces a Human-AI partnership, where Mergent collaborates with reviewers to refine vague feedback, maintain consistent tone and focus, and evaluate the quality of reviewer comments to help them improve over time. Finally, the Post-Review Feedback phase delivers a concise summary of the review outcomes and provides clear next-step suggestions for the PR author.

By integrating Large Language Models (LLMs) into a human-in-the-loop workflow, Mergent bridges the gap between human expertise and AI efficiency. The system ultimately aims to reduce review time, improve feedback quality, and establish a more structured and intelligent PR review ecosystem.

1.2. High-Level System Architecture & Components

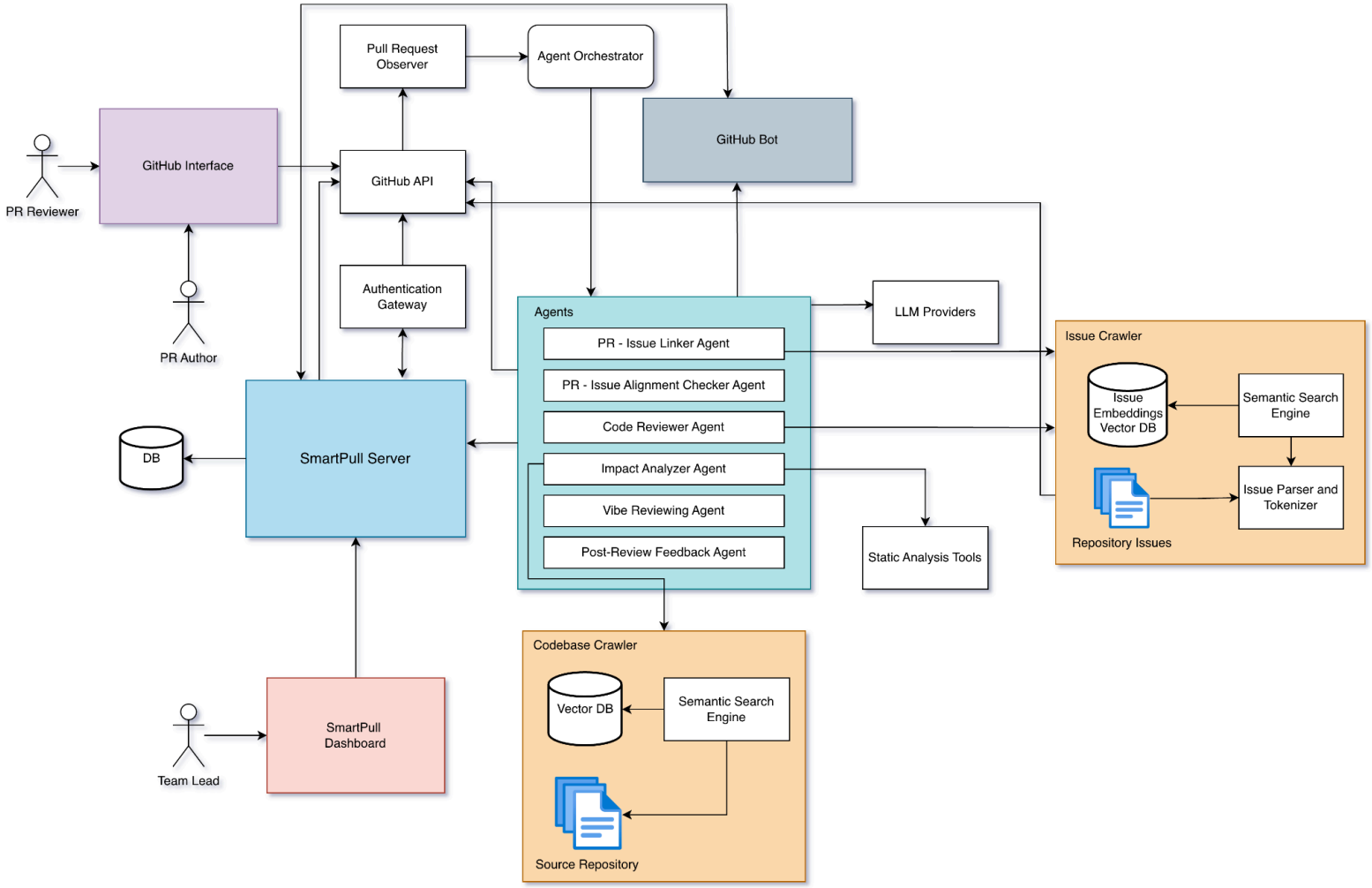

Figure 1. High-Level Architecture Diagram of Mergent

The diagram shows how Mergent operates around the GitHub ecosystem. GitHub events which are creation of pull requests are received through the GitHub API and workflow of the Pull Request Observer is triggered. The Mergent Server acts as the central controller, connected to the Authentication Gateway, Database, and Agent Orchestrator.

The Agent Orchestrator manages multiple agents, including the PR–Issue Linker, PR-Issue Alignment Checker, Code Reviewer, Impact Analyzer, Vibe Reviewer, and Post-Review Feedback Agent. These agents communicate with LLM Providers and Static Analysis Tools while accessing contextual data from the Codebase Crawler, Issue Crawler, and Vector Databases.

Processed outputs are then sent back to GitHub through the GitHub Bot, completing the review cycle.

1.3. Constraints

1.3.1. Implementation Constraints

- **Availability of suitable training data**: Limited training data for pull requests, issues, and review comments reduces the system's ability to generalize across different projects.
- **Reliance on LLM APIs:** Mergent's performance depends on LLM quality and external API access rather than local compute power.
- **Integration with external platforms:** GitHub or JIRA APIs have rate limits and restricted control over the review process, which limits functionality.
- **Limited control over workflows**: Since Mergent works within existing development environments and version control systems, it can only extend workflows through APIs or bots, not modify them directly.

1.3.2. Economic Constraints

- **Cost of LLM API calls:** Frequent use of LLMs for code analysis and feedback generation increases operational expenses.
- **Infrastructure and hosting costs:** Cloud hosting, API integrations, and deployment infrastructure contribute to budget limitations.

1.3.3. Sustainability Constraints

- **High energy and resource usage:** Frequent LLM calls and large-scale analyses can increase energy consumption and operational costs.
- **Scalability limitations:** The system must remain lightweight to avoid performance degradation and excessive infrastructure demand as usage grows.

1.3.4. Time Constraints

- **Limited development time:** As a two-semester capstone project, development time is finite.
- **Competing project activities:** Research, testing, and documentation also require time, restricting the implementation of additional features and integrations beyond the core scope.

1.3.5. Privacy Constraints

- **Handling of sensitive information:** Mergent processes real code, commit messages, comments, issues, and pull requests that may contain proprietary data, requiring strict confidentiality.

- **External API interactions:** Data passed to external services must be protected to prevent leaks outside the authorized environment.

1.4. Professional and Ethical Issues

The development of this project requires adherence to professional and ethical conduct within the team and in interactions with external collaborators. All team members are expected to contribute responsibly, avoid plagiarism in both code and documentation, and ensure that all work produced is original and properly attributed. Tasks and responsibilities of the project should be distributed fairly to the Mergent team to maintain a balanced and respectful working environment.

Since the project will be tested or demonstrated in collaboration with an external company, confidentiality must be strictly maintained. Any proprietary information, internal documentation, or source code provided by the partner organization must not be shared outside the project group. All exchanges must comply with the company's security and privacy policies, and the ownership of both the project code and any shared assets must be clearly respected.

Professionalism also requires transparent communication, respectful teamwork, and adherence to deadlines. Ethical awareness throughout the project ensures that the development process remains fair, trustworthy, and aligned with academic and industry standards.

1.5. Standards

The project follows UML 2.5.1 for system and process modeling to maintain consistency and clarity in design diagrams [3]. IEEE 830-1998 standards are applied for requirements specification to ensure structured and well-documented system requirements [4]. In addition, general IEEE software engineering practices are followed to support maintainability, reliability, and quality throughout development.

2. Design Requirements

2.1. Functional Requirements

2.1.1. PR-Issue Linkage and Suggestion

The system shall manage the connection between Pull Requests (PRs) and their corresponding issues.

- **Detect Linkage**
  The system shall automatically detect whether a submitted PR is linked to an issue.
- **Suggest Existing Issue**
  If a PR is not linked to an issue, the system shall search all existing issues and suggest the most relevant one to be linked.
- **Generate New Issue**
  If a PR is unlinked and no relevant existing issue is found, the system shall be able to automatically generate and link a new issue based on the PR's content and context.

2.1.2. PR-Issue Alignment Validation

The system shall validate that the content of a PR aligns with the requirements of its linked issue.

- **Verify Connection**
  The system shall analyze the code changes in the PR to verify that it correctly addresses the description in the linked issue.
- **Categorize Alignment**
  The system shall classify the alignment into one of the following four categories:

1. Exact: The PR fully implements the changes described in the issue.
2. Missing: The PR fails to implement the required changes.
3. Tangling: The PR includes additional, unrelated changes not mentioned in the issue.
4. Missing & Tangling: The PR both fails to address the issue's requirements and adds unrelated work.

2.1.3. AI-Generated Review Comments

The system shall be able to automatically generate code review comments.

- **Generate Contextual Comments**
  The system shall generate clear, specific, and actionable AI-powered review comments based on the code changes within the PR.

- **Provide Actionable Suggestions**

  The comments should provide concrete suggestions for improvement, such as adding a necessary null check to a function.

2.1.4. PR Enrichment with Risk & Impact Insights

The system shall enrich Pull Requests (PRs) with data that helps reviewers prioritize their focus by analyzing potential risks to the production environment.

- **Identify Impact**

  The system shall highlight the files and modules affected by the changes in the PR.
- **Assess Historical Risk**

  The system shall flag changes in code areas that are historically bug-prone or have been part of past failures.
- **Analyze Production-Level Risk**

  The system shall analyze the potential impact of the PR on the live production environment. This includes flagging changes that could affect critical services, introduce security vulnerabilities, or negatively impact system performance.
- **Provide Risk Score**

  The system shall calculate and display a simple, easy-to-understand risk score that consolidates historical and production-level risk factors for the PR.

2.1.5. Vibe Reviewing

The system shall enable human-AI collaboration to ensure consistent, clear, and high-quality review feedback.

- **Refine Reviewer Feedback**

  The system shall analyze human review comments and rewrite vague or repetitive ones into precise, actionable messages.
- **Ensure Consistency**

The system shall maintain a uniform tone and review style across all reviewers to avoid conflicting or unclear feedback.

- **Enhance Human Comments**

  When a vague comment is detected, the system should suggest a more specific and actionable rewrite to help the reviewer provide more effective feedback.

2.1.6. Post-Review Feedback

The system shall generate a unified post-review summary that consolidates AI and human feedback.

- **Summarize Key Outcomes**

The system shall compile all review comments into a single, concise summary highlighting major findings and resolutions.

- **Provide Follow-Up Actions**

  The system shall generate clear, prioritized next steps (e.g., add tests, refactor modules, document changes).
- **Deliver Author Insights**

  The system shall provide structured feedback as a report to the PR author that includes improvement areas and overall review quality insights.

2.2. Non-Functional Requirements

2.2.1. Usability

- Shall integrate seamlessly with one version control system, such as GitHub or BitBucket.
- AI feedback and scores should be clear and easy to understand.

2.2.2. Reliability

- AI suggestions and alignment checks should be accurate and trustworthy.
- Analysis should be consistent for similar PRs to ensure predictable results.
- Shall gracefully handle failures from external services like LLM providers without crashing.

2.2.3. Performance

- The AI analysis part of the PR analysis workflow must be completed within a maximum of 5 minutes to avoid slowing down the review cycle.
  - Issue Agent and Alignment Agent should respond within 2 minutes.
  - AI comment generation should take no more than 2 minutes.
  - PR enrichment should take no more than 5 minutes.
- The vibe reviewing phase, which depends on human collaboration, should be responsive; hence, the response time should be under 15 seconds.

2.2.4. Security

- All user source code and PR data shall be encrypted in transit and at rest.
- Mergent shall authenticate via a secure Authentication Gateway using OAuth 2.0 or JWT-based tokens.

- User source code must not be used to train general AI models without the user's explicit consent.

2.2.5. Availability

- The core service shall maintain a high uptime of at least 99% for daily development workflows.
- Shall be fault-tolerant to prevent a single agent failure from causing a system-wide outage.
- Shall have backup and disaster recovery plans to minimize downtime.

2.2.6. Scalability

- Mergent should handle at least 50 concurrent PR analyses across different repositories without degradation.
- Each AI agent shall be scaled independently to maintain performance under high load.

3. Feasibility Discussions

3.1. Market & Competitive Analysis

The market for AI-assisted pull request (PR) review tools has grown rapidly in recent years, driven by the increasing complexity of modern software systems and the rising cost of slow or inconsistent code reviews. Existing tools generally address isolated parts of the PR workflow—automated comments, static analysis, risk flags, or PR descriptions—but few deliver an integrated, end-to-end solution. Mergent was intentionally designed to fill this gap.

Existing Solutions and Their Limitations

Qodo (formerly CodiumAI) focuses heavily on code generation, automated PR descriptions, and applying actionable code suggestions [5]. Qodo Merge offers strong AI review comments but lacks deeper enrichment signals, such as historical bug-prone area detection or repository-specific risk scoring. It also provides partial support for reviewer assistance (PR chat) [5]. However, it lacks true reviewer coaching and structured post-review feedback.

CodeRabbit acts as a near-autonomous AI reviewer, producing line-by-line comments, static code-graph analysis, and generating docstrings or sequence diagrams [6]. While powerful, CodeRabbit assumes issues are already linked and focuses primarily on code automation rather than collaborative review quality. It does not evaluate reviewer comments, provide enrichment beyond code-graph analysis, or offer post-review guidance.

LinearB addresses organizational bottlenecks, AI adoption metrics, and automation for PR routing [7]. It includes “Knowledge Share” to expose junior reviewers to expert reviews.[7]. However, it does not perform deep PR–issue alignment, risk-based enrichment, or post-review author support. Its reviewer-development features inspired Mergent’s Vibe Reviewing phase, yet these capabilities remain limited when applied directly to PR content.

CodeScene delivers maintainability and technical-debt–oriented insights, using hotspots, change coupling, and code health profiles [8]. Although it offers strong structural analysis, it does not assist with issue linkage, semantic alignment, reviewer coaching, or author-side summaries. It focuses on code health, not on review process quality.

CodeAnt AI combines compliance checks, secret scanning, and policy enforcement with AI-generated PR summaries [9]. However, it does not perform semantic issue alignment, risk scoring, reviewer-comment evaluation, or author-side feedback. It focuses more on organizational governance than collaborative review improvement.

GitHub Copilot offers AI reviews and inline suggestions[10]. However, its PR support is limited to summarization and basic recommendations. It does not provide enrichment signals, reviewer training, or post-review consolidation. Its strength is code-generation and autonomous patching—not improving human review quality.

How Mergent Differentiates Itself

Unlike existing tools, Mergent combines multiple advanced capabilities into a single, cohesive workflow. It performs end-to-end PR intelligence: automatic issue linkage or generation, deep PR–issue alignment, AI review comments, PR risk enrichment, reviewer coaching, and post-review author support. Each of these phases has competitors, but no tool unifies them in a structured, human-in-the-loop system.

Most existing tools treat the AI as a replacement reviewer. Mergent treats AI as a co-pilot—enhancing reviewers, not replacing them. This includes evaluating reviewer comment quality, flagging unclear feedback, standardizing review tone, and offering the author a summarized digest after the review. Such features are not present in current market offerings.

Furthermore, Mergent emphasizes contextual PR intelligence, incorporating ideas from commit risk analysis, change-impact analysis, and repository-specific patterns—something competitors either partially implement (CodeScene) or ignore entirely.

3.2. Academic Analysis

Evaluating ReLink for Traceability Link Recovery in Practice [11]
This study presents an experience report on applying ReLink, a semi-automated tool for recovering links between pull requests (PRs) and issues, within both open-source and industrial environments (e.g., Beko's Connecta platform). Utilizing classical NLP techniques—specifically Word2Vec embeddings combined with TF-IDF weighting and heuristic rules—ReLink achieved a top-5 accuracy of 0.86 and a Mean Reciprocal Rank (MRR) of 0.84. However, the authors note that the system relies on static semantic similarity and lacks the contextual understanding of deep language models like BERT. Mergent addresses this limitation in its PR-Issue Linkage phase by employing Large Language Models (LLMs) that can interpret ambiguous text and context, offering a significant improvement over ReLink's non-contextual embedding approach.

Enhancing Pull Request Reviews: Leveraging Large Language Models to Detect Inconsistencies Between Issues and Pull Requests [12]
Işık et al. investigated the efficacy of LLMs in detecting inconsistencies between issue descriptions and PR code changes using a manually labeled dataset from the HuggingFace Transformers repository. The study classified alignments into four categories: exact, missing, tangling, and missing & tangling. By evaluating models such as Llama-3.1-405B and GPT-4o, the researchers found that providing a comprehensive context window—comprising the issue text, PR description, and the full code diff—resulted in the highest performance (Weighted F1: 0.619). These findings directly shape the design of Mergent's Alignment Checker Agent, confirming that analyzing the intersection of natural language intent and actual code implementation is crucial for accurate validation.

Commit Guru: Analytics and Risk Prediction of Software Commits [13]
Commit Guru introduces a language-agnostic tool designed to predict risky commits by analyzing 13 distinct change-level metrics, such as file entropy, lines added, and developer experience. Unlike complex deep learning models, it utilizes a logistic regression model trained on historical bug-inducing commits to provide interpretable risk probabilities. The tool's success in highlighting risky changes through statistical modeling supports Mergent's PR Enrichment strategy. Mergent adopts a similar philosophy by calculating a consolidated "Risk Score," but it enhances this approach by combining these metadata-driven analytics with semantic code understanding to prioritize reviewer attention more effectively.

DeputyDev - AI Powered Developer Assistant: Breaking the Code Review Logjam through Contextual AI to Boost Developer Productivity [14]
This paper proposes DeputyDev, an AI assistant that addresses review bottlenecks through a multi-agent architecture featuring specialized roles such as Security, Performance, and Code Maintainability agents. A key innovation is the use of a

"Reflection Pattern," where agents generate an initial review and then self-reflect to reach a consensus, significantly improving output quality. Mergent mirrors this architectural style in its Agent Orchestrator, which manages distinct agents (e.g., Impact Analyzer, Vibe Reviewer) to ensure modularity. By decomposing the review process into specialized subtasks, Mergent aims to replicate the structured and high-confidence feedback demonstrated by DeputyDev.

Back to the Basics: Rethinking Issue-Commit Linking with LLM-Assisted Retrieval [15]

The EasyLink approach challenges traditional deep learning methods for traceability by introducing a two-stage pipeline: initial retrieval using vector databases (FAISS) followed by LLM-assisted reranking. In realistic evaluation settings, this method achieved a Precision@1 of 75.03%, outperforming state-of-the-art models like EALink by a significant margin. The study demonstrates that while vector retrieval captures similarity, LLMs are essential for bridging the "semantic gap" between issue intent and commit messages. Mergent incorporates this hybrid logic into its PR-Issue Linker Agent, utilizing efficient vector search for candidate generation and LLMs for precise semantic verification.

Automated Code Review Using Large Language Models with Symbolic Reasoning [16]

This research proposes a hybrid code review approach that combines fine-tuned LLMs with a symbolic reasoning layer, specifically a structured knowledge map of common Python defect patterns. Experiments using the CodeXGLUE dataset showed that this hybrid model, particularly when using GraphCodeBERT, achieved a 27% improvement over baseline LLMs, reaching an F1-score of 0.381. These results highlight that LLMs alone often struggle with logical consistency. Consequently, Mergent's Code Reviewer Agent is designed to interface with static analysis tools, ensuring that generative feedback is not just linguistically fluent but also grounded in verified logic and established coding standards.

Can Large Language Models Replace Static Analysis Tools? [17]

This article investigates the capability of models like GPT-4 to replace traditional static analysis tools (e.g., SonarQube) across categories like security, performance, and code style. The findings indicate that while LLMs are proficient at explaining issues, they struggle with precise line localization and adherence to domain-specific or custom rules. The authors conclude that LLMs should function as augmenters rather than replacements. Mergent aligns with this conclusion by integrating static analysis tools into its workflow; the Code Reviewer Agent uses LLMs to interpret and prioritize static analysis findings, thereby reducing false positives while retaining the precision of rule-based systems.

From Overload to Insight: Bridging Code Search and Code Review with LLMs [18]

This study argues that the future of code review lies in a "human-in-the-loop" process where AI serves as a collaborative partner rather than a fully autonomous reviewer. It suggests utilizing diverse knowledge bases—including historical data, CI logs, and documentation—to create conversational agents capable of reasoning over multiple data sources. This concept is the primary inspiration for Mergent's Vibe Reviewing phase. Instead of focusing solely on bug detection, this phase emphasizes refining the tone and clarity of human comments and facilitating better communication, effectively turning the AI into a coach for the human reviewer.

Combining Large Language Models with Static Analyzers for Code Review Generation [19]

This paper explores three strategies for merging static analysis (KBS) with LLMs (LBS): Data-Augmented Training (DAT), Retrieval-Augmented Generation (RAG), and Naive Concatenation (NCO). The experiments demonstrated that RAG—injecting static analysis findings directly into the LLM's inference prompt—yielded the highest accuracy by providing real-time, rule-based context without the need for expensive retraining. Mergent implements this RAG strategy within its agent workflows, feeding outputs from static analysis tools into the context window of its LLM agents to generate reviews that are both contextually aware and technically precise.

Enhanced Code Reviews using Pull Request based Change Impact Analysis [20]

This study introduces the "Change Impact Detector" (CHID), a tool that assesses the risk of PRs by integrating static dependency analysis with historical repository metrics. By constructing call graphs to trace ripple effects (using PageRank) and analyzing factors like code churn and bug frequency, CHID provides a visualized risk score that improved reviewer awareness in focus group studies. Mergent leverages these specific techniques in its Impact Analyzer Agent. The agent assesses not just the immediate code changes but the broader impact on the system's dependency graph to compute a consolidated risk score, directly aiding reviewers in prioritizing high-risk modifications.

4. Glossary

PR: Pull Request.
Issue: A task explanation, bug report, or feature request describing required work in the project.
AI: Artificial Intelligence.
LLM: Large Language Model.
Human-in-the-Loop: A design strategy where human reviewers collaborate with AI systems to improve accuracy, tone, and review quality.
JWT: JSON Web Token, a secure token format standard that is used for authentication and authorization of users.
OAuth 2.0: A standard protocol for secure user authentication and access delegation.
Static Analysis: Automated code inspection that identifies errors or issues without the need to run the program.
Repository (Repo): The project's source code, which is maintained through a version control system.
Code Diff: The set of added, removed, or modified lines of code in a pull request.
Version Control Systems: Systems to control, organize and track the different versions of computer files. These systems are commonly used for software development.

5. References

[1] U. Cihan et al., "Automated Code Review in Practice," 2025 IEEE/ACM 47th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP), Ottawa, ON, Canada, 2025, pp. 425-436, doi: 10.1109/ICSE-SEIP66354.2025.00043.

[2] C. Liu, H. Y. Lin and P. Thongtanunam, "Too Noisy To Learn: Enhancing Data Quality for Code Review Comment Generation," 2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR), Ottawa, ON, Canada, 2025, pp. 236-248, doi: 10.1109/MSR66628.2025.00043.

[3] OMG, Unified Modeling Language (UML), Version 2.5.1. Needham, MA, USA: Object Management Group, Dec. 2017. [Online]. Available: https://www.omg.org/spec/UML/2.5.1

[4] "IEEE Recommended Practice for Software Requirements Specifications," in IEEE Std 830-1998 , vol., no., pp.1-40, 20 Oct. 1998, doi: 10.1109/IEEESTD.1998.88286.

[5] Qodo, “Qodo: AI-Assisted Code Suggestions and PR Review Automation,” qodo.ai. Accessed: Nov. 25, 2025. [Online]. Available: https://www.qodo.ai/

[6] CodeRabbit, “AI Code Review and Automated Pull Request Comments,” coderabbit.ai. Accessed: Nov. 25, 2025. [Online]. Available: https://coderabbit.ai/

[7] LinearB, “Engineering Analytics and Workflow Automation Platform,” linearb.io. Accessed: Nov. 25, 2025. [Online]. Available: https://linearb.io/

[8] CodeScene, “Code Health and Change Impact Analysis Platform,” codescene.com. Accessed: Nov. 25, 2025. [Online]. Available: https://codescene.com/

[9] CodeAnt AI, “AI-Powered Developer Compliance and PR Analysis,” codeant.ai. Accessed: Nov. 25, 2025. [Online]. Available: https://codeant.ai/

[10] GitHub, “GitHub Copilot: AI Pair Programmer,” github.com. Accessed: Nov. 25, 2025. [Online]. Available: https://github.com/features/copilot

[11] A. Yaşa et al., "Evaluating ReLink for Traceability Link Recovery in Practice," 2025 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER), Montreal, QC, Canada, 2025, pp. 80-90, doi: 10.1109/SANER64311.2025.00016.

[12] A. T. Işık, H. Kübra Çağlar and E. Tüzün, "Enhancing Pull Request Reviews: Leveraging Large Language Models to Detect Inconsistencies Between Issues and Pull Requests," 2025 IEEE/ACM Second International Conference on AI Foundation Models and Software Engineering (Forge), Ottawa, ON, Canada, 2025, pp. 168-178, doi: 10.1109/Forge66646.2025.00027.

[13] C. Rosen, B. Grawi, and E. Shihab, ‘Commit guru: analytics and risk prediction of software commits’, in Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering, Bergamo, Italy, 2015, pp. 966–969.

[14] V. Khare, V. Saini, D. Sharma, A. Kumar, A. Rana, and A. Yadav, ‘DeputyDev -- AI Powered Developer Assistant: Breaking the Code Review Logjam through Contextual AI to Boost Developer Productivity’, arXiv [cs.SE]. 2025.

[15] H. Huang et al., ‘Back to the Basics: Rethinking Issue-Commit Linking with LLM-Assisted Retrieval’, arXiv [cs.SE]. 2025.

[16] B. Icoz and G. Biricik, ‘Automated Code Review Using Large Language Models with Symbolic Reasoning’, arXiv [cs.SE]. 2025.

[17] H. Cui, ‘Can large language model replace static analysis tools’, in International Conference on Computer Network Security and Software Engineering (CNSSE 2024), 2024, vol. 13175, pp. 320–325.

[18] N. Rao, B. Vasilescu, and R. Holmes, ‘From Overload to Insight: Bridging Code Search and Code Review with LLMs’, in Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering, New York, NY, USA: Association for Computing Machinery, 2025, pp. 656–660.

[19] I. Jaoua, O. B. Sghaier, and H. Sahraoui, ‘Combining Large Language Models with Static Analyzers for Code Review Generation’, arXiv [cs.SE]. 2025.

[20] I. S. Göçmen, A. S. Cezayir, and E. Tüzün, ‘Enhanced code reviews using pull request based change impact analysis’, Empir. Softw. Eng., vol. 30, no. 3, May 2025.